CS & IT ENGINEERING



Algorithms

Dynamic Programming



Lecture No.- 06

By- Aditya sir

Recap of Previous Lecture



- Topic
- Topic

LCS
mcm

Topics to be Covered



- Topic
- Topic
- Topic

mcm

sos



About Aditya Jain sir

1. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
2. Represented college as the first Google DSC Ambassador.
3. The only student from the batch to secure an internship at Amazon. (9+ CGPA)
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program, specialization in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10
9. Joined Dream11 as a Data Scientist
10. Have mentored 12,000+ students & working professions in field of Data Science and Analytics
11. Have been mentoring & teaching GATE aspirants to secure a great rank in limited time
12. Have got around 27.5K followers on Linkedin where I share my insights and guide students and professionals.



Telegram Link for Aditya Jain sir: https://t.me/AdityaSir_PW



Topic : Dynamic Programming

Matrix Chain Multiplication/Product (MCM):

#Q. Given two matrices A and B of size m×n and n×p.

How many scalar multiplication are reqruied in $A_{m\times n} * B_{n\times p} = C$?



Topic : Dynamic Programming

Case 1:

Square Matrices:

$$A_{n\times n} * B_{n\times n} = C_{n\times n}$$

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} * \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}$$



Topic : Dynamic Programming

$c_{11} = a_{11} \times b_{11} + a_{12} \times b_{21}$

$c_{12} = a_{11} \times b_{12} + a_{12} \times b_{22}$

$c_{21} = a_{21} \times b_{11} + a_{22} \times b_{21}$

$c_{22} = a_{21} \times b_{12} + a_{22} \times b_{22}$

Every element = 2

Hence, total scalar multiplication = $2 \times (2 \times 2) = 2 \times 2 \times 2$



Topic : Dynamic Programming

Case 2:

Non-Square Matrices:

$A_{m \times n} * B_{n \times p} = C_{m \times p}$

- Hence, for all m×p element.

  Total scalar multiplication = $n \times (m \times p) = m \times n \times p$



Topic : Dynamic Programming

Important Result:

$$A_{m\times n} * B_{n\times p} = C_{m\times p}$$

- Total number of scalar multiplication = $m \times n \times p$



Topic : Dynamic Programming

MCM Problem Statement:

- Given a chain of computable non-square matrices, it is required to multiply them together to get a final resultant matrix.



Topic : Dynamic Programming

Example:

$n = 3$

$[A_{2\times10}, B_{10\times50}, C_{50\times20}] = (ABC) = Z_{2\times20}$



Topic : Dynamic Programming

Parenthesizing Problem:

ABC = (1) $(A * B) * C \Rightarrow 3000$

(2) $A * (B * C) \Rightarrow 10{,}400$



Topic : Dynamic Programming

(1) (A * B) * C

How many Scalar multiplication?

(i) (A * B) = 2 × 10 × 50

(ii) (A * B) * C = 2 × 50 × 20

So, total scalar multiplication for (A * B) * C are

= 2 × 10 × 50 + 2 × 50 × 20

= 1000 + 2000 = 3000



Topic : Dynamic Programming

(2) A * (B * C)

(i) (B * C) = 10 × 50 × 20

(ii) A * (B * C) = 2 × 10 × 20

So, total scalar multiplication for A * (B * C) are

= 10 × 50 × 20 + 2 × 10 × 20

= 1000 + 400

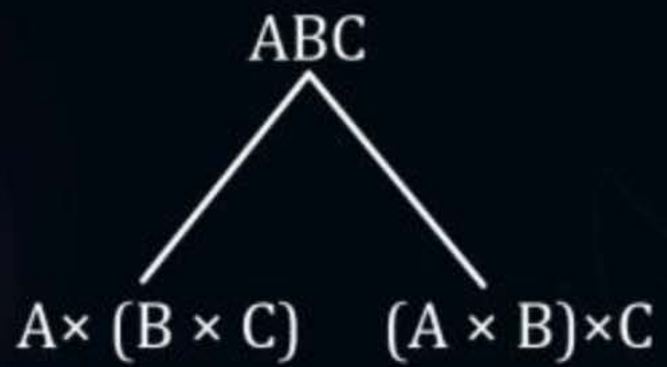
= 10400



Topic : Dynamic Programming

MCM objective function

to minimize the number of scaler multiplication.





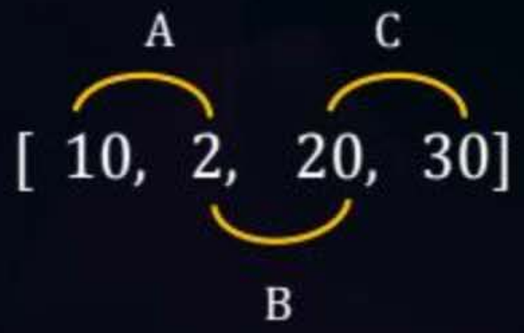
Topic : Dynamic Programming

DP based approach:-

V. Imp

- Given a chain of n matrixes (A1, A2 .... An) where matrix Ai is of Dimension $P_i - 1 * P_i$.

  The MCM problem is to fully parenthesize the chain such that the total number of scalar multiplication are minimized.





Topic : Dynamic Programming

#Q. How many ways to multiply chain n matrices?

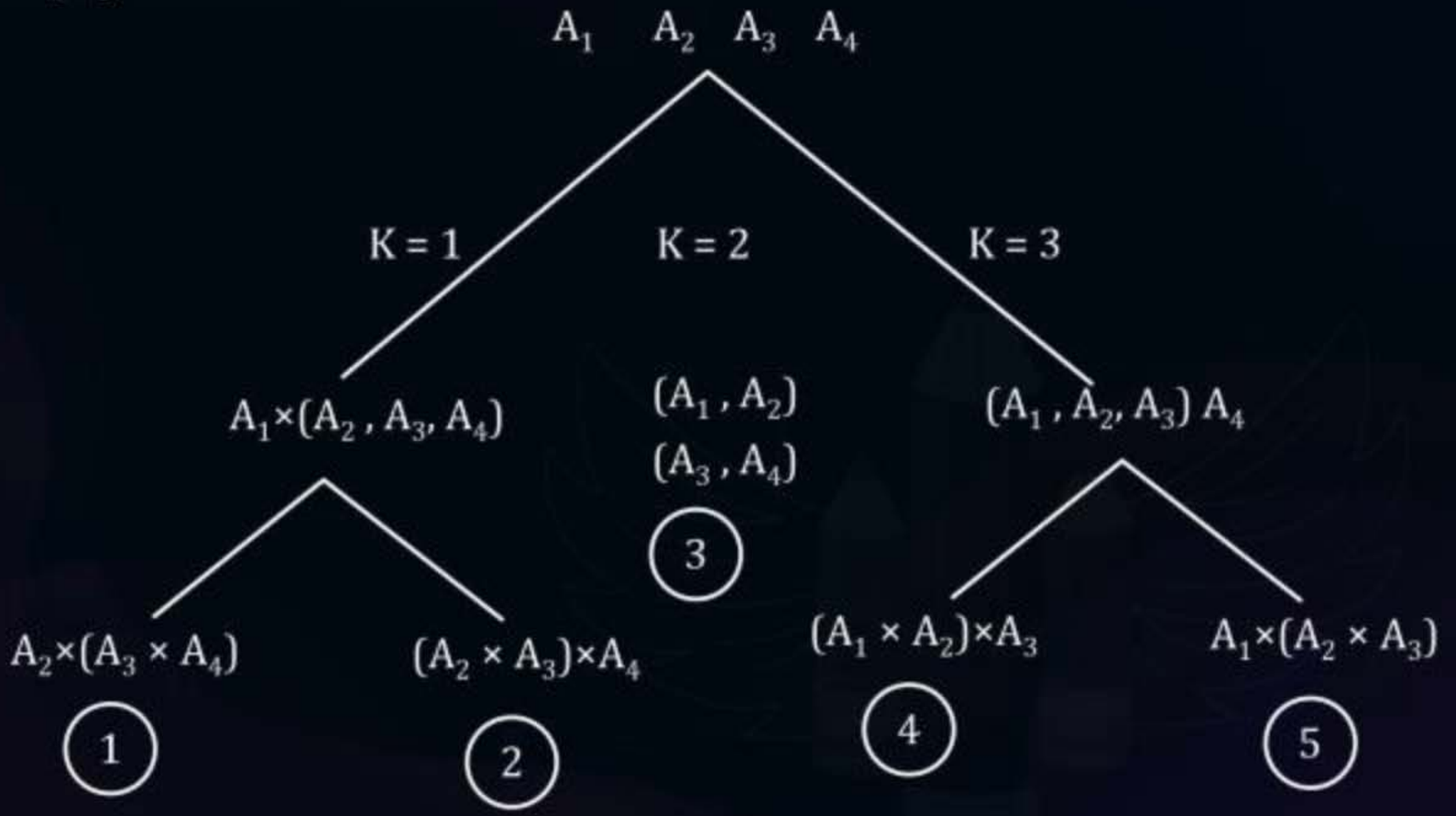
K = 1 K=2

↓ ↓

ABC

(1) A*(B*C)

(2) (B*C)*C



5 ways



Topic : Dynamic Programming

Generalized Results:-

Total no. of ways to multiply a chain of (n + s) matrix $\Rightarrow$ Catalan number

E.g. N = 3

$\downarrow$

4 matrix

$$= \frac{1}{4} \times {}^{6}C_3$$

$$= \frac{1}{4} \times \frac{6 \times 5 \times 4 \times 3}{3! \times 3!}$$

$$= \frac{6 \times 5}{5} = 5$$

$$\boxed{\frac{1}{(n+1)} \times {}^{2n}C_n}$$

n +1

n $\Rightarrow$ n-1



Topic : Dynamic Programming

n Matrix $\rightarrow \frac{1}{n} \times^{2(n-1)} C_{(n-1)}$



Topic : Dynamic Programming

Deamination of DP based Recurrence for men:

Let the resultant matrix Ai j be of product

$(A_i * Ai+1 * A_{i+2} \ldots\ldots\ldots A_j)$

Chain → Result Ai j

$A_1 \times A_2 \times A_3 \ldots\ldots\ldots A_n \Rightarrow A_{1\times n}$ $\quad\quad$ k = (1 to n–1)



Topic : Dynamic Programming

Any optimal Parameterization must split the chain about the matrix A_k & A_{k+1} rule that the number of scaler multiplications are minimum.



Topic : Dynamic Programming

DP Recurrence

Let m[i, j] represent the number of scalar multiplication to get the resultant matrix Ai j.

$$[A_i \times A_{i+1} \ldots\ldots\ldots A_k * \ldots\ldots A_j] \Rightarrow A\ i\ j \rightarrow M\ [i, j]$$

Split at k

$$(A_i \times A_i + \ldots\ldots A_k) \times (A_{k+1} \ldots\ldots\ldots A_j) \quad \text{Where} \rightarrow\ i \leq k \leq (j-1)$$

$m\ [i\ ,\ k]$ $\qquad$ $m\ [k+1,\ j]$



Topic : Dynamic Programming

For $\forall$ $i \le k \le j-1$

$$M[i, j] = \min \{m[i,k] + m[k+1, j] + P_{i-1} * P_k * P_j\}$$

(v.v.imp)

$(A_i \ldots\ldots\ldots\ldots\ldots A_k) * (A_{k+1} \ldots\ldots\ldots\ldots\ldots A_j)$

$P_{i-1}*P_i$ $\quad$ $P_{k-1}*P_k$

$P_{i-1}*P_k$

P_k*P_{k+1} $\quad$ $P_{j-1}*P_j$

P_k*P_j

R $\quad$ $*$ $\quad$ R

$P_{i-1}*P_k$ $\quad$ P_k*P_j

$(P_{i-1}*P_k*P_j)$

A_i

$(P_{i-1}*P_j)$



Topic : Dynamic Programming

Example:

M[1,6]

$(A_1 \rightarrow A_6)$ ↓ $K = 3$

$A_{16} = (A_1 \ A_2 \ A_3) \quad (A_4 \ A_5 \ A_6)$

$(A_1 \ldots\ldots\ldots\ldots A_3) \quad (A_4 \ldots\ldots\ldots\ldots A_6)$

2×3 $P_0\times P_1$ | M [1,3] | 5×8 $P_2\times P_3$

8×4 $P_3\times P_4$ | M [4,6] | 7×6 $P_5\times P_6$

2×8 $P_0\times P_3$

8×6 $P_3\times P_6$

Given,

$A_1 \rightarrow P_0 \times P_1 \rightarrow 2\times3$

$A_2 \rightarrow P_1 \times P_2 \rightarrow 3\times5$

$A_3 \rightarrow P_2 \times P_3 \rightarrow 5\times8$

$A_4 \rightarrow P_3 \times P_4 \rightarrow 8\times4$

$A_5 \rightarrow P_4 \times P_5 \rightarrow 4\times7$

$A_6 \rightarrow P_5 \times P_6 \rightarrow 7\times6$

: $[P_0 \ldots\ldots\ldots P_n]$

Number of both Resultants = $P_0 * P_3 \times P_6$

$= 2 \times 8 \times 6$

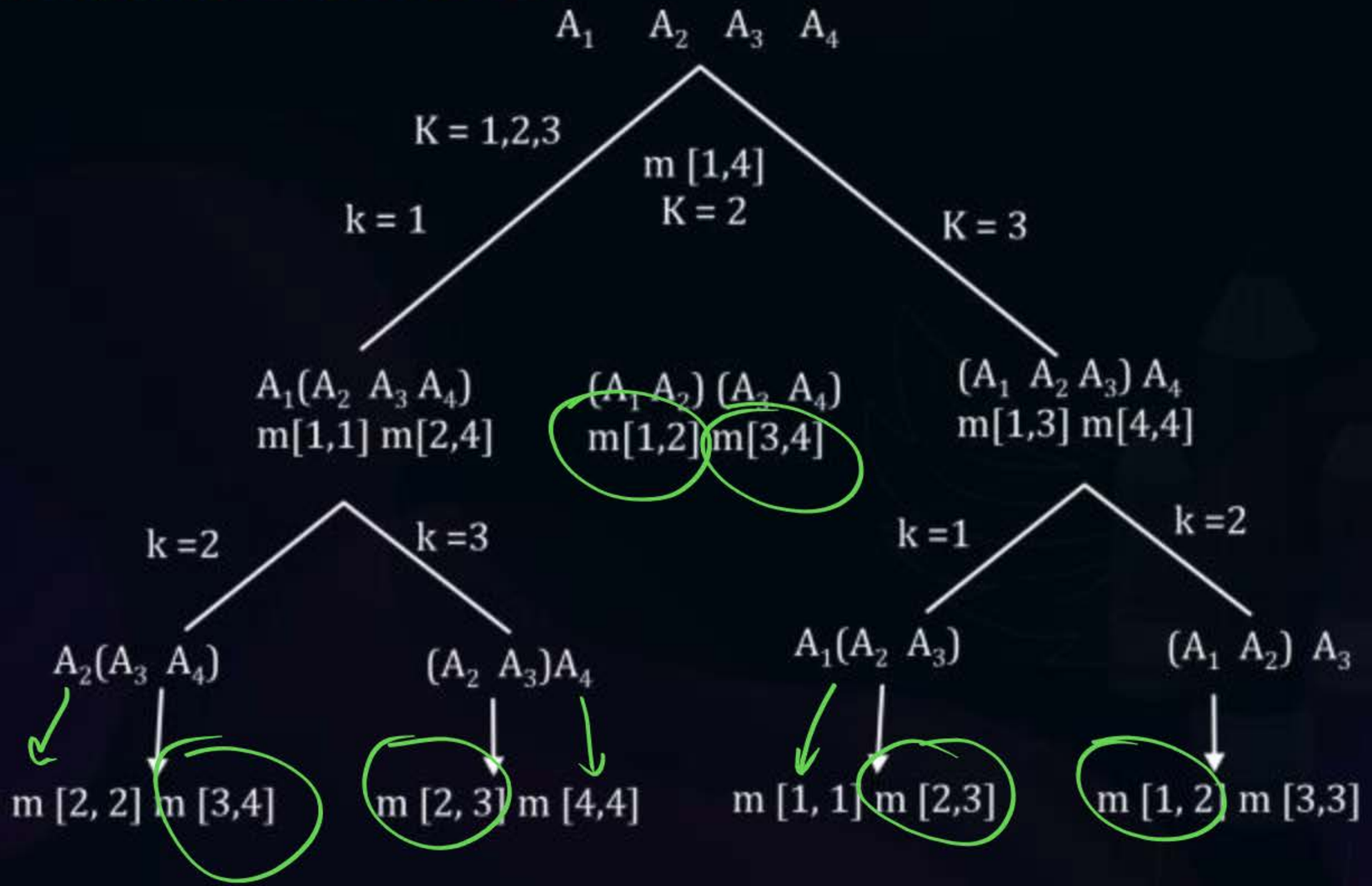
i =1, k = 3, j = 6

$(A_1 A_2 A_3 A_4)$



Topic : Dynamic Programming

Are there overlapping Subproblems?

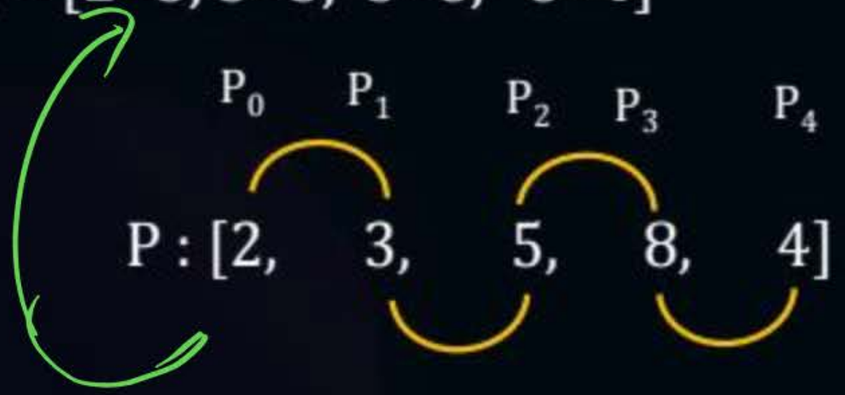




Topic : Dynamic Programming

Top Down: Recursion Approach

E.g. $A_1\ A_2\ A_3\ A_4 = [2\times3,\ 3\times5,\ 5\times8,\ 8\times4]$

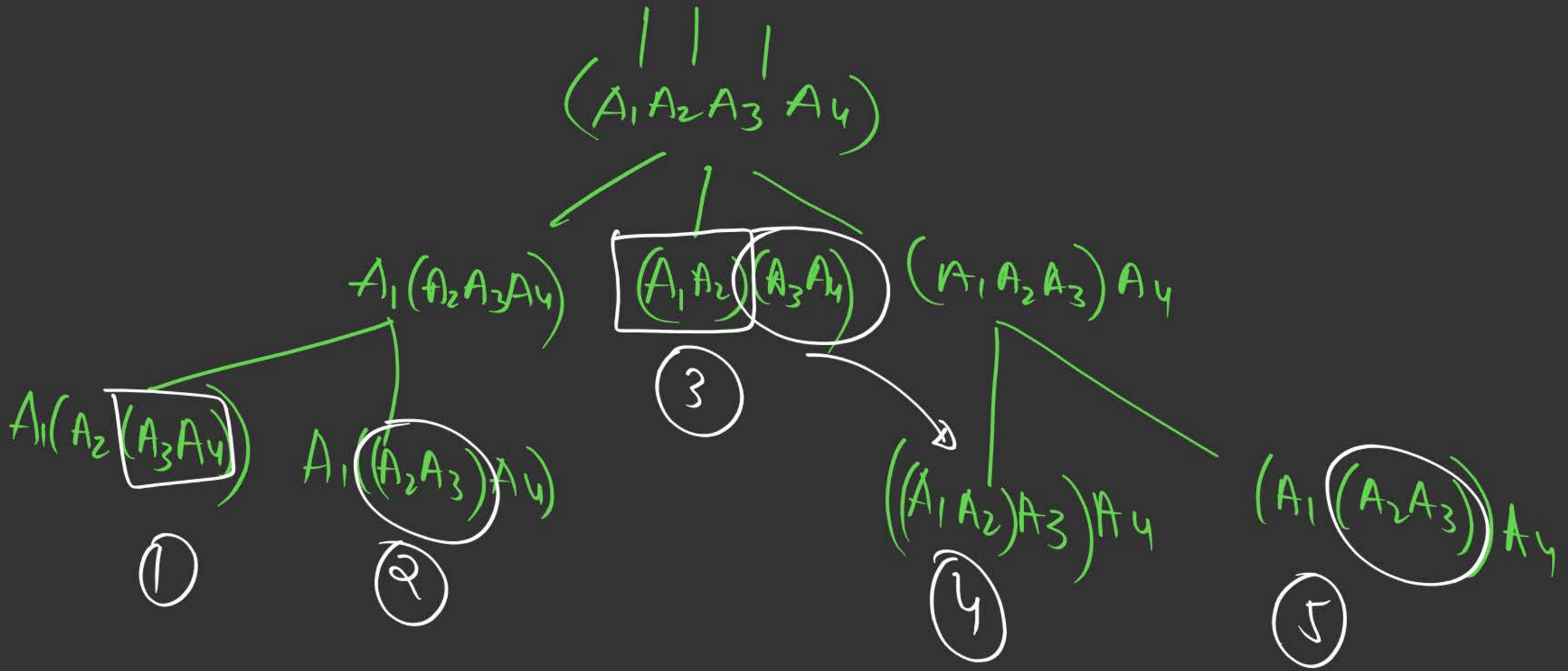




Topic : Dynamic Programming

#Q. The minimum no. of scaler multifaction required to ~~null~~ mul The chain $A_1A_2A_3A_4$ = ?

$(A_1A_2A_3A_4)$


$(A_1 A_2 A_3 A_4)$
$A_1(A_2 A_3 A_4)$
$(A_1 A_2)(A_3 A_4)$
$(A_1 A_2 A_3) A_4$
$A_1(A_2(A_3 A_4))$
$A_1((A_2 A_3) A_4)$
$((A_1 A_2) A_3) A_4$
$(A_1 (A_2 A_3)) A_4$
1
2
3
4
5

Soln:- $((A_1 A_2) A_3) A_4$

① ② ③

2 3 5 8 4

$A_1 \rightarrow 2 \times 3$
$A_2 \rightarrow 3 \times 5$
$A_3 \rightarrow 5 \times 8$
$A_4 \rightarrow 8 \times 4$

$A_1 A_2 \rightarrow 2 \times 3 \times 5$

$(R\ A_3) \rightarrow 2 \times 5 \times 8$

$R_{2 \times 8}\ A_4 \rightarrow 2 \times 8 \times 4$

$30 + 80 + 64 = 174$ ✓

Bottom Up: Code



Topic : Dynamic Programming

Algorithm Matrix-Chain-Product (p)

1. n ←length [p] – 1
2. for i ← 1 to n
3. do m[i ,i] ← 0
4. for *l* ← 2 to n (*l* is the chain length.)
5. for i ← 1 to n – *l* + 1
6. J ← i + *l* – 2
7. m[i, j] ← ∞

$P_0 \rightarrow P_n$

} diagonals

$(A_1 - - - A_n)$

i j

A_{ij}



Topic : Dynamic Programming

V-imP

```
8.  for k ← i to j – 1
9.      q ← m[ i , k] + m[k + 1, j] + P_{i-1} P_k P_j
10.     if (q < m [i , j])
11.     then m[i , j] ← q
12.     s[i, j] ← k
13. return m and s
```

(Splitting point)

A i j

$i \le k \le j-1$

$(A_1, A_5) \Rightarrow s[1,5]$

k → soln to that subprob



Topic : Dynamic Programming :(DP)

Explanation:

$m[i, j] = (A_i \; A_{i+1} \ldots A_j)$

$m[i, j] = A_{ij} \rightarrow$ ~~sealer~~ Scaler multiplication

$A_1 \longrightarrow A_4$

$l = 2$

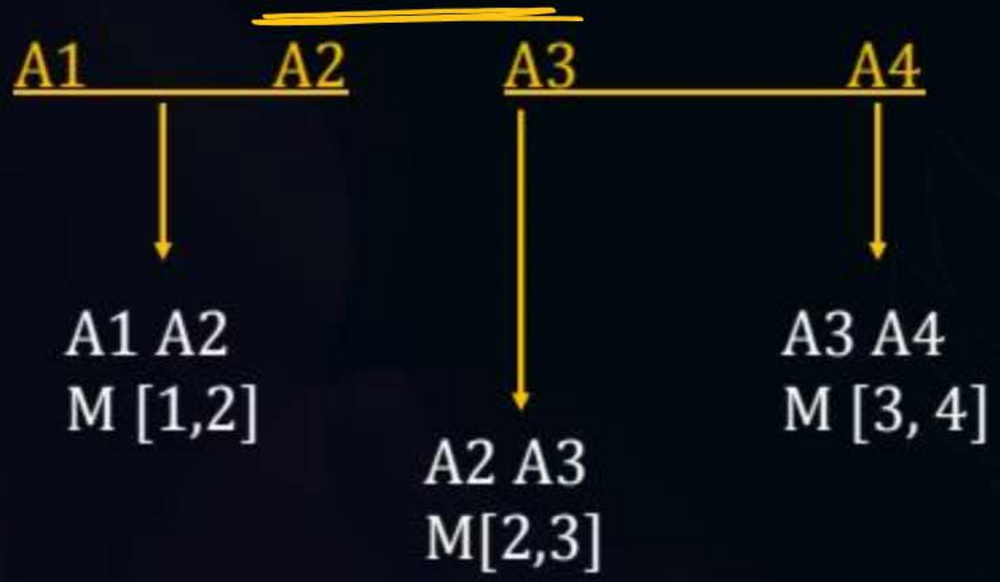
	0	1	2	. . . .	n
0	0				
1		0			
2			0		
.				0	
.					0
n					



Topic : Dynamic Programming :(DP)

L→ length of the matrix chain

E.g. l = 2 (Chain of length 2)





Topic : Dynamic Programming :(DP)

$A_1, A_2 \ldots\ldots\ldots\ldots A_n$ Bonus Chain l

$A_1 A_2 A_3 A_4$

$l = 2 \rightarrow n - 2 + 1 = n - 1$

$l = 3 \rightarrow n - 3 + 1 = n - 2$

$l = 4 \rightarrow n - 4 + 1 = n - 3$

$n - l + 1$

$A_1, A_2 \ldots An_1 \quad An$

$(A_1 \longrightarrow A_n)$

$J = i + l - 2$



Topic : Dynamic Programming :(DP)

Complexity Analysis of Top– down MCM (DP approach)

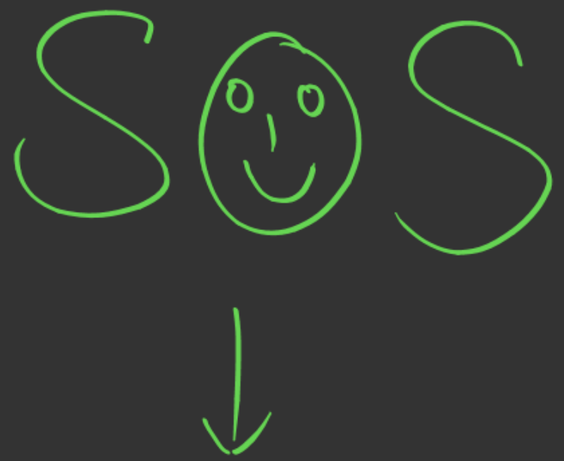
Assume n → no. of matrix

1. Time Complexity: $O(n^3)$ → for every length chain and for every break-up point in it.

2. Space complexity:- $O(n^2)$

   $M[n \times n]$

   Brute force / Enumeration: TC : $\Omega(2^n)$


SOS



Topic : Dynamic Programming :(DP)

6. Sum of subset: (SOS)

Problem Statement:- Given a set of n-elements (integers) and also another element (integer) 'M'.

The sum of subsets (SOS) problem is to determine If there exits a subset of the given elements whose sum equals to 'M'.

↓

Yes/No True/False



Topic : Dynamic Programming :(DP)

SOS→ Decision Problem

Optimization problems (min/max)

1. Bellman – Ford→ Minimize path cost (SSSP)
2. Floyd Warshall → Minimize path cost (APSP)
3. 0/1 knapsack → Maximize Profit
4. Longest common Subsequence (LCS) → Maximize length of common subsequence.
5. Matrix chain multiplication (MCM) → Minimize scalar multiplication



Topic : Dynamic Programming :(DP)

SOS→ Very similar to 0/1 knapsack problem:

Eg.1. N = 5

1 2 3 4 5

A =[50, 30, 10, 40, 20]

M = 50

Subsets	Index
{50}	{1}
{30, 20}	{2, 5}
{10, 40}	{3, 4}

} Yes / True



Topic : Dynamic Programming :(DP)

Enumeration

Brute Force:-

Check every subset→ → subset

TC: $O(2^n)$



Topic : Dynamic Programming :(DP)

DP: SOS→ Top-Down (Recurrence of SOS DP)

Target Sum

n, A[Ai..... An], M, X[1.....n]

Let SOS (n, m) represent the solution to the SOS, with n elements and target sum (M).

SOS (n, m) = True/ False = 0/1



Topic : Dynamic Programming :(DP)

SOS (n, m) = T {There exits a subset from n elements that sum to M}

= F { There does not exits}

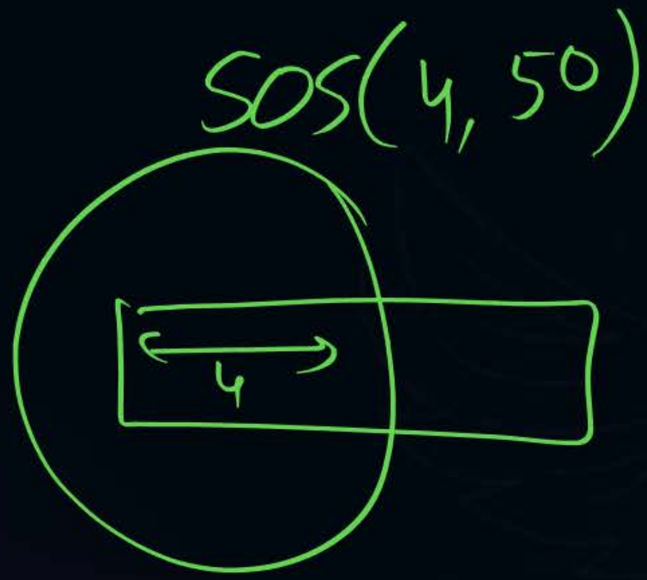


F or R → F

F or T → T

T or F → T

T or T → T





Topic : Dynamic Programming :(DP)

Ai >= 0

SOS (n, M) = SOS (n–1, M) , An > M {skip/ reject An→ x}

SOS (n–1, M)

or

SOS (n –1, M –An)

logical OR

An ≤ M

Choice

An ✓

An ✗

It we can get to sum M from either taking An or Rejecting An



Topic : Dynamic Programming :(DP)

Boundary case

or

Base condition

V. V. imp

SOS (n, m) = False , n = 0 , M > 0

SOS (n, m) = True , M = 0 , n ≥ 0

n=0 & M=0 → True







Topic : Dynamic Programming :(DP)

Explanation

SOS (n, m) = True ?

M = 0, n = 5

Take decision of each of n elements
(not mandatory to take or leave them)

SOS

n = 0, m = 0

{True}

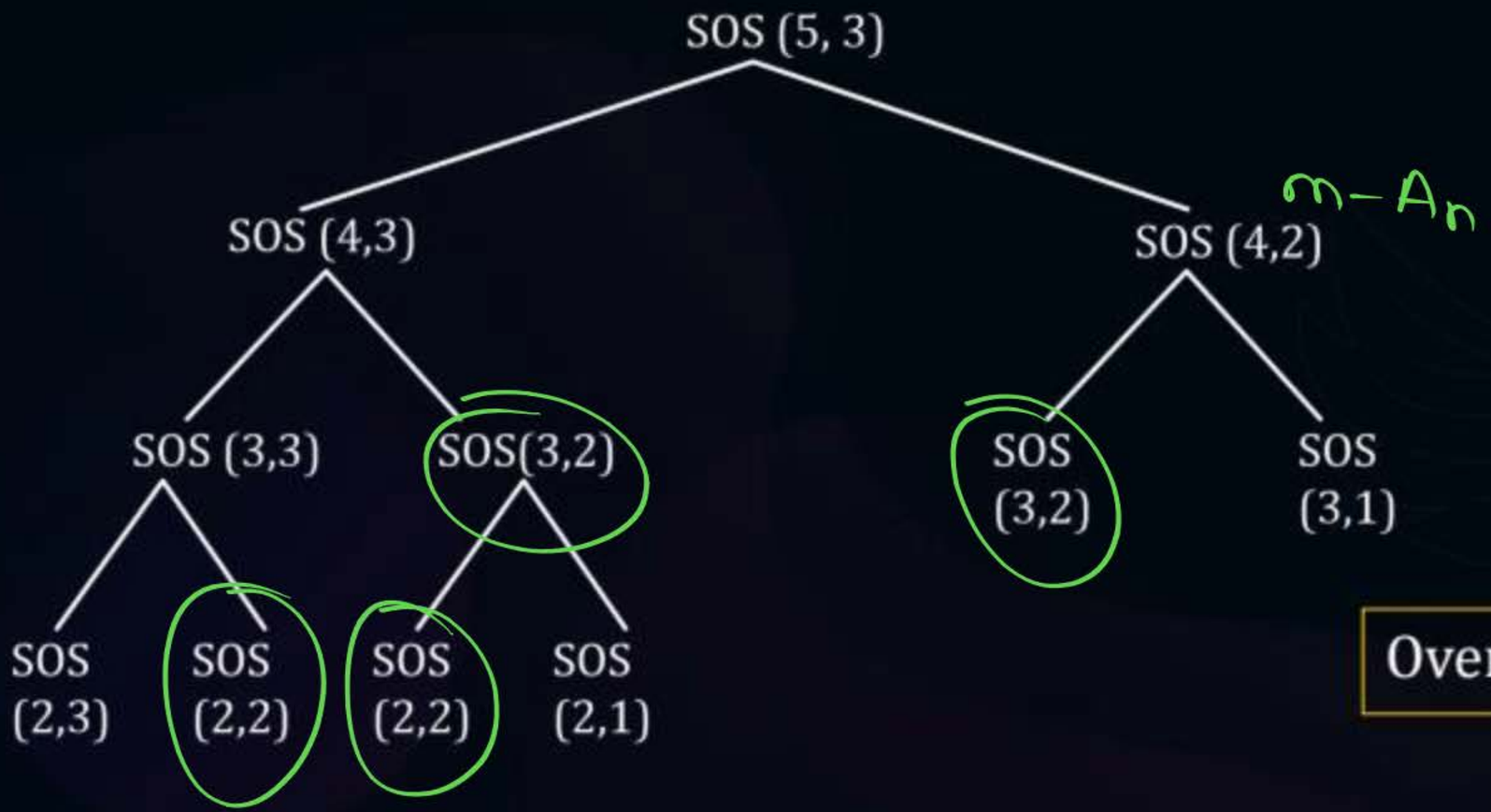
$n=5$, $A[1,1,1,1,1]$ $M=3$



Topic : Dynamic Programming :(DP)

TOP – Down

E.g.:- n =5, A =[1, 1, 1, 1, 1] M = 3



Overlapping subproblems



Topic : Dynamic Programming :(DP)

SOS → Bottom Up (Tabulation Method)

Given:- n, m x [0, ……n 0…..m]

X[i,j] = True /False

A particular I,j call form x[,]

Whether there exits a subset from the first 'i' elements that sum to 'j'.

E.g.2. A = [2, 8, 4, 11, 9]

n= 5,

m = 6

True



Topic : Dynamic Programming :(DP)

Ai	X	0	1	2	3	4	5	6
0	0	T	F	F	F	F	F	F
2	1	T	F	T	F	F	F	F
8	2	T	F	T	F	F	F	F
4	3	T	F	T	F	T	F	T
11	4	T	F	T	F	T	F	T
9	5	T	F	T	F	T	F	T

Final ans



Topic : Dynamic Programming :(DP)

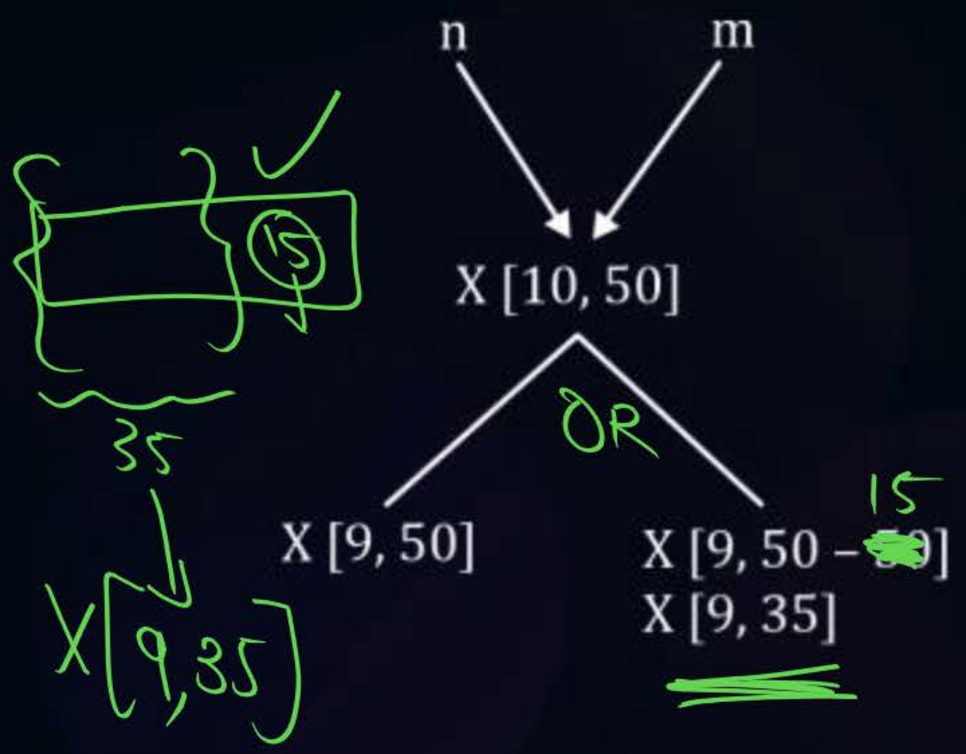
Imp Shortest

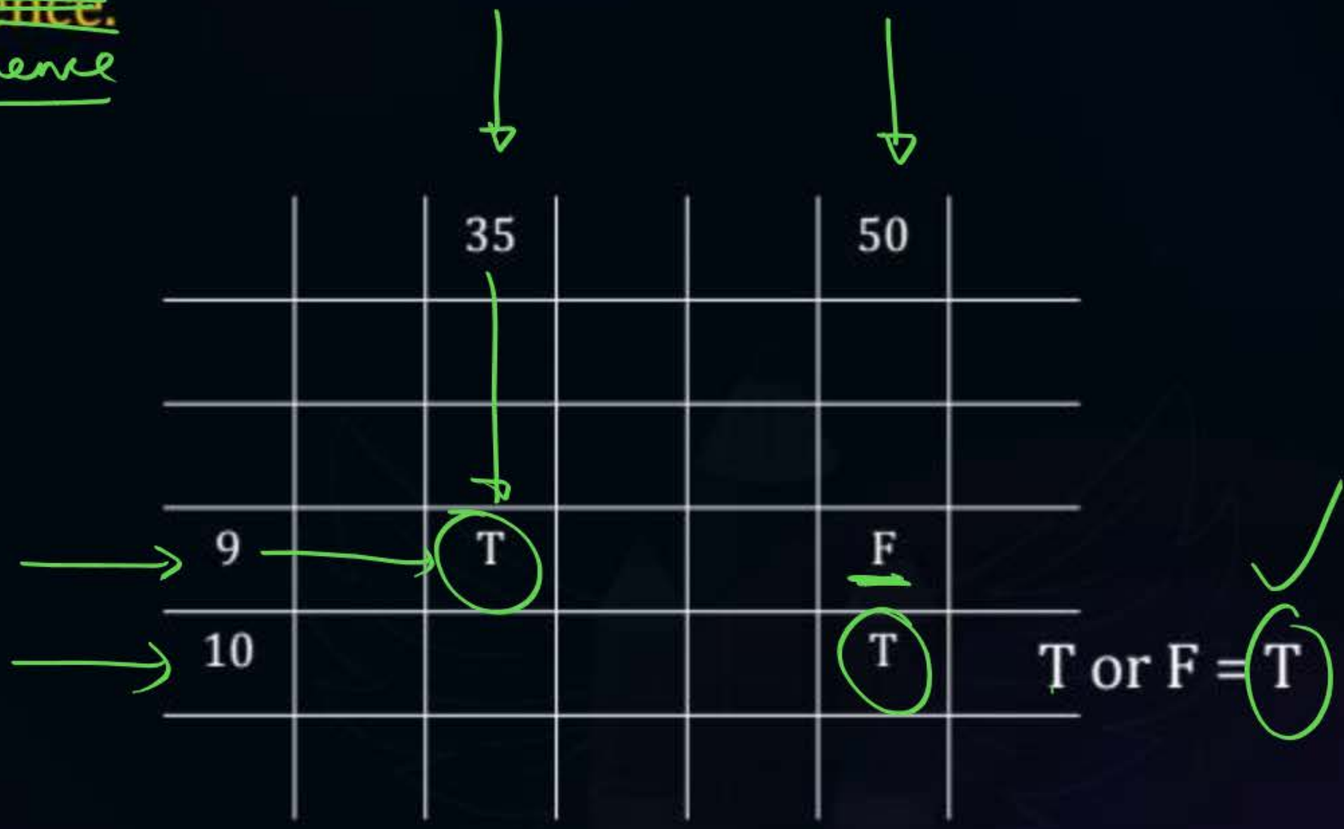
In A [I , j] = True then the entire column ‘j’ can be set to True.

Topic : Dynamic Programming :(DP)

For large example, how to use ~~recoherence~~. recurrence

Bonus





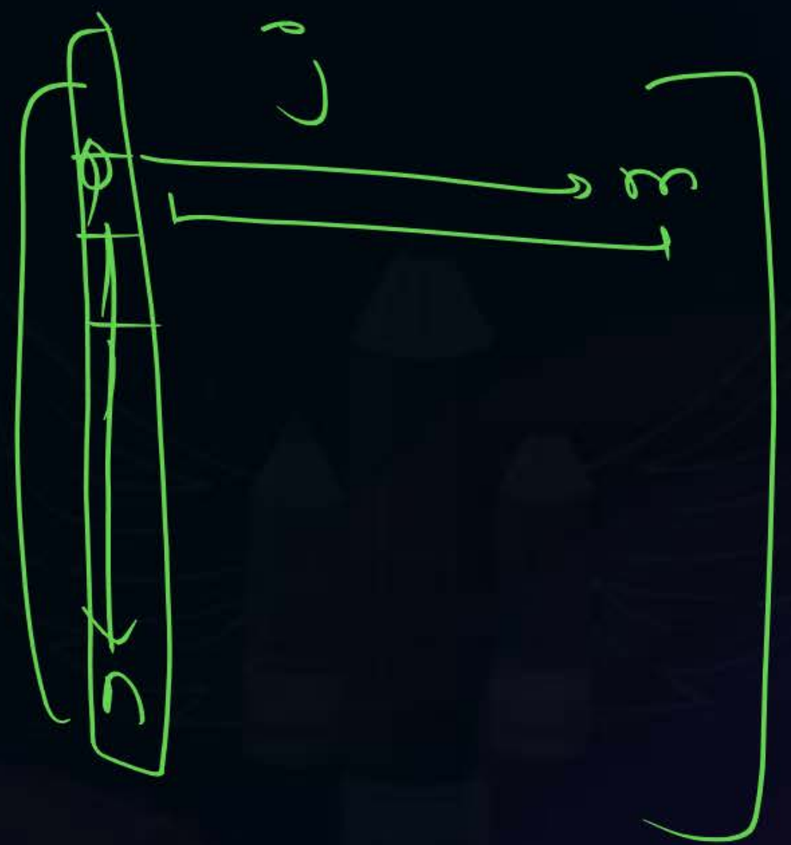
Topic : Dynamic Programming :(DP)

DP: SOS: Tabulation (Bottom -up) Code

H.W

```
Algo SOS (n, M, A)          A [1.....n]
{
    X [0....n,   0.....M] → Auxiliary Space
        for i = (0 to n):
    {
        for j = (0 to m)
        {
            if (i ≥ 0 and j =0)
            {
                X [i, j] = True
            }
            else  if [I = 0 and j > 0]
            }
            else if (A [i] > j)
            {
                x [I,j] = x [i–1 , j]
            }
            Choice  Ai ≤ j
        else
            {
                x [i,j] = (x[i–1, j] or X[i–I, j –A [i]])
            }
        }
    }
}
```

A(i,j)=F





Topic : Dynamic Programming :(DP)

Complexity Analysis of Bottom- up (DP) Approach of SOS:

1. Time complexity : $O(n*m)$
2. Space complexity : $O(n*m)$

Note:-

If M is v.v. large $\rightarrow O(2^n)$ then TC $\rightarrow O(n*2^n)$



2 mins Summary

- Topic
- Topic
- Topic
- Topic
- Topic

MCM
SOS





THANK - YOU